

Controladores en Express

GUÍA DE ARQUITECTURA DE BACKEND - UNIDAD 5

1. El Patrón Controlador (MVC)

Los controladores son el cerebro lógico de tu API. Su función estricta es actuar como intermediarios (la capa "C" en el patrón Modelo-Vista-Controlador). Cuando un router intercepta una petición, no debe ejecutar código, debe delegarla inmediatamente al controlador correspondiente.

Las 3 Responsabilidades de un Controlador:

1. Recibir los *inputs* del cliente (`req.body` , `req.params`).
2. Orquestrar el procesamiento de datos (delegando al Modelo o a los Servicios).
3. Ensamblar y despachar el *output* (`res.json` , códigos HTTP de estado).

2. Patrones de Diseño Físico

A nivel de hardware lógico, puedes estructurar tus controladores de tres formas según el stack de tu proyecto:

- **1. Como Objeto Literal:** Creas un `const userController = {}` y le metes funciones asíncronas adentro. Ideal para proyectos ligeros.
- **2. Como Clase ES6:** Creas un `class UserController` y lo instancias al exportar (`module.exports = new UserController()`). Permite usar herencia y encapsulamiento (POO pura).
- **3. Funciones Individuales:** Usas `exports.metodo = async () => {}` directamente. Permite importar funciones específicas, optimizando el consumo de memoria.

3. Arquitectura "Thin Controller" (La Capa de Servicios)

Si metes algoritmos matemáticos complejos, conexiones a base de datos y criptografía directamente adentro de tu controlador, vas a generar un "Fat Controller" que es in-testeable e in-mantenible. Para evitar esto, se crea una capa de **Servicios**.

Componente	Responsabilidad (I/O)
Router	Agente de tráfico. Escucha la URL y dirige al Controlador.
Controlador	Capa de entrada/salida HTTP. Formatea la respuesta (200 OK, 400 Error).
Servicio	Contiene la lógica pesada de negocio. Hashea passwords, hace cálculos, se comunica con el Modelo de la BD. No sabe qué es "req" o "res" .

4. Manejo de Errores Avanzado

Llenar tu código de bloques `try/catch` en cada endpoint genera redundancia y desperdicia I/O de disco. La arquitectura moderna exige un enfoque centralizado:

- **Controlador Base:** Una clase "padre" (`BaseController`) que estandariza las salidas (`successResponse` y `errorResponse`).
- **Wrapper Asíncrono (`asyncHandler`):** Una función de orden superior que intercepta promesas rechazadas y las envía directamente al middleware de errores, ahorrándote tipear el `try/catch` en cada ruta.

5. El Firewall de Entrada: Validación con Joi

Un controlador nunca debe procesar *payloads* crudos directamente de la red. La validación es la primera línea de defensa de memoria.

Con librerías como **Joi**, compilas un esquema (Schema) matemático que audita el JSON entrante. Si los datos no cumplen con las reglas de tipo (`String`, `Int`) o longitud, un middleware bloquea la petición (HTTP 400) **antes** de que llegue a consumir ciclos de CPU de tu Controlador.

```
// Definición del hardware de validación (Joi Schema)
const productSchema = Joi.object({
  name: Joi.string().required().min(3).max(100),
  price: Joi.number().required().min(0),
  category: Joi.string().required()
});

// En tu Router, interceptas antes del controlador:
router.post('/producto', validate(productSchema),
productController.create);
```

6. Debugging y Testing

Al aislar la lógica en Controladores y Servicios, el testing se vuelve predecible. Se usan motores como **Jest** y **Supertest** para:

- Simular peticiones HTTP (Unit Testing) enviando payloads controlados.

- Validar los *status codes* exactos devueltos por tu API (201 Created, 404 Not Found).
- Comprobar si el ruteador detona correctamente el firewall de validación cuando mandas datos corruptos.

Documento generado a partir de los conceptos de la Unidad 5 - Controladores y Arquitectura de API en Express